

FICHE DE TD 3/3 • DIFFICILE

Informatique • Tle TI

Modélisation dynamique des systèmes d'information avec UML (cas d'utilisation et séquence)

Module 3 : Systèmes d'information et bases de données

Cette fiche propose des exercices de synthèse mobilisant la modélisation UML dynamique (cas d'utilisation et séquences) dans des contextes réalistes complexes, exigeant rigueur formelle, justification des choix de conception et cohérence inter-diagrammes.

EXERCICE 1 Système de Gestion de Concours – Cas d'utilisation

On modélise le système d'information d'un « Concours National » gérant l'inscription, l'organisation des épreuves, la correction et la proclamation des résultats. Les acteurs identifiés sont : Candidat, Administrateur, Jury, Correcteur, Système de Paiement (externe).

Les fonctionnalités principales sont :

- S'inscrire (nécessite paiement des frais).
- Modifier son dossier (avant clôture).
- Imprimer sa convocation.
- Composer (passer l'épreuve).
- Corriger une copie (anonymat obligatoire).
- Saisir les notes.
- Délibérer (calcul moyennes, affectation mentions).
- Publier les résultats.
- Gérer les réclamations (post-publication).
- Administrer les paramètres du concours (dates, centres, coefficients).

Règles métier :

1. L'inscription inclut obligatoirement le paiement.
 2. Un candidat peut étendre son inscription par une demande d'aménagement (handicap), gérée par l'Administrateur.
 3. La correction est une spécialisation de « Évaluer » ; la délibération en est une autre.
 4. La gestion des réclamations étend la publication des résultats (déclenchée si erreur signalée).
 5. L'Administrateur généralise les acteurs « Gestionnaire Utilisateurs » et « Gestionnaire Concours ».
1. Diagramme : Dessiner le diagramme de cas d'utilisation complet. Respecter la

notation UML 2.5 (acteurs, cas d'utilisation, frontières, relations `include`, `extend`, `generalization` avec stéréotypes et conditions d'extension). (4 pts)

2. Scénarios : Rédiger le scénario nominal et deux scénarios alternatifs pour le cas d'utilisation « S'inscrire », en explicitant les points d'extension et d'inclusion. (3 pts)
3. Analyse critique : Justifier pourquoi la relation entre « Corriger une copie » et « Évaluer » est une généralisation et non une inclusion. Quelle contrainte d'anonymat cela impose-t-il sur la ligne de vie « Copie » dans un diagramme de séquence associé ? (3 pts)

EXERCICE 2 Traitement d'une Inscription – Diagramme de séquence

On considère le cas d'utilisation « S'inscrire » du système précédent. Le flux nominal est :

1. Le Candidat remplit le formulaire sur l'interface Web.
2. Le système valide les données (format, pièces jointes).
3. Le système crée un objet `DossierCandidat` (état BROUILLON).
4. Le Candidat lance le paiement via l'API `SystemePaiement`.
5. Le `SystemePaiement` retourne OK ou ECHEC.
6. Si OK : le `DossierCandidat` passe à VALIDE, une `Convocation` est générée (PDF), email envoyé.
7. Si ECHEC : le `DossierCandidat` passe à ANNULE, notification envoyée.

Contraintes techniques : Le paiement est asynchrone (callback). L'envoi d'email est délégué à un service `NotificationService`. La génération de PDF utilise une librairie `PDFGenerator`.

1. Diagramme nominal : Réaliser le diagramme de séquence UML 2.5 complet. Montrer : lignes de vie (acteurs, objets, services), messages synchrones/asynchrones, messages de retour, création/destruction d'objets (`create`, `destroy`), fragments combinés `alt` (paiement), `loop` (tentatives paiement max 3), `par` (génération PDF && envoi email). (5 pts)
2. Gestion d'exception : Ajouter au diagramme un fragment `opt` ou `alt` pour gérer l'indisponibilité du `NotificationService` (`retry` + log erreur sans bloquer la validation du dossier). (2 pts)
3. Cohérence : Expliquer comment la relation `include` (« S'inscrire » → « Payer ») identifiée à l'exercice se manifeste structurellement dans ce diagramme de séquence (référence, décomposition, ou inline). (2 pts)

EXERCICE 3 Refactoring et Traçabilité – Étude de cas critique

On vous soumet deux extraits de modèles UML produits par une équipe junior pour un module « Bibliothèque Numérique » (Prêt/Retour).

Extrait 1 – Diagramme de cas d'utilisation (textuel) : Acteur *Membre*. Cas d'utilisation : « Emprunter ouvrage », « Réserver ouvrage », « Prolonger prêt », « Payer pénalité ». Relations : « Emprunter » *include* « Vérifier quota » ; « Prolonger » *extend* « Emprunter » (condition : pas de réservation en attente) ; « Payer pénalité » *extend* « Retourner » (condition : retard).

Extrait 2 – Diagramme de séquence (textuel) pour « Emprunter ouvrage » : *Membre* → *UI* : *choisirOuvrage()* *UI* → *Catalogue* : *chercher(isbn)* *Catalogue* → *UI* : *Ouvrage* *UI* → *Membre* : *afficherDispo()* *Membre* → *UI* : *confirmerEmprunt()* *UI* → *Preteur* : *emprunter(membre, ouvrage)* *Preteur* → *Preteur* : *verifierQuota()* // Appel interne *Preteur* → *Stock* : *decrements()* *Preteur* → *Preteur* : *creerPret()* *Preteur* → *UI* : *Pret* *UI* → *Membre* : *confirmation()*

1. Erreurs UC : Identifier trois erreurs conceptuelles ou notationnelles majeures dans l'Extrait 1 (relations mal choisies, acteurs manquants, périmètre flou). Proposer la correction formelle pour chacune. (4 pts)
2. Erreurs Séquence : Identifier deux incohérences entre l'Extrait 2 et les règles UML 2.5 (typage messages, fragments manquants, responsabilité mal placée). Corriger le diagramme textuellement. (3 pts)
3. Traçabilité : La relation *include* (« Emprunter » → « Vérifier quota ») est-elle correctement reflétée dans l'Extrait 2 ? Si non, redessiner (textuellement) la partie concernée du diagramme de séquence pour faire apparaître explicitement cette inclusion comme une interaction structurée (fragment *ref* ou sous-diagramme). (3 pts)
4. Extension : Proposer le fragment de séquence (texte ou esquisse) correspondant à l'extension « Payer pénalité » déclenchée lors du « Retourner », en gérant le cas où le paiement est refusé (dossier bloqué). (2 pts)

EXERCICE 4 Modélisation Avancée – Système Distribué "Click & Collect"

Un commerce met en place un service « Click & Collect » : commande web, préparation en magasin, retrait en drive. Acteurs : *Client*, *SystemeWeb*, *PreparateurMagasin*, *HoteDrive*, *SystemePaieement*, *SystemeStock*. Règles :

- Commande : *Panier* → *Paieement* (3D Secure) → *Confirmation*.
- Préparation : *Notif Preparateur* → *Prélèvement Stock* (réservation) → *Emballage* → *Statut PRET*.
- Retrait : *Client arrive* → *Scan QR Code* → *Vérification identité* → *Remise colis* → *Clôture*.
- Exceptions : *Rupture stock pendant prépa* (annulation partielle + remboursement), *Client absent* (mise en attente 48h puis annulation).

1. Diagramme UC global : Produire le diagramme de cas d'utilisation principal. Faire apparaître au moins une généralisation d'acteurs, une inclusion, une extension avec condition, et la frontière du système. (3 pts)
2. Diagramme de séquence "Retrait Drive" : Modéliser le scénario nominal + l'alternative « Client absent » (boucle attente 48h). Utiliser les fragments `alt`, `loop`, `break`, et montrer la destruction de l'objet `Commande` lors de l'annulation finale. (5 pts)
3. Analyse d'outils : Dans StarUML ou ArgoUML, comment garantir la cohérence entre le nom du cas d'utilisation « Retirer commande » (UC) et le nom de l'interaction `sd RetirerCommande` (Séquence)? Citer la propriété UML (métamodèle) qui lie un `UseCase` à son `Interaction` réalisatrice. (2 pts)